CIS 2368 FALL 2025 Final Project

Recipe Manager and cooking helper

For your final project you will be building a full stack web application following a standard 3 tier architecture:

UI – REST API Services – Database

This application will help users to plan and manage recipes / meals.

Overview:

The user has the option to manage the ingredients, the recipes, and the recipe-ingredients. Thus, you will need to create at least the following tables:

recipe, ingredient, **recipeingredient**.

For all tables, the id should be the primary key, and autogenerated. Pick appropriate data types for each column.

The **recipe** table should at minimum contain the following columns:

id, name, instructions

The **recipeingredient** table should have the following columns:

id, recipeid, ingredientid, amount

The **ingredient** table should have the following columns:

id, ingredientname, totalamount

The **recipeingredient** table, has **foreign keys** to both the **ingredient** and **recipe** table. The user should be able to perform CRUD on recipes and the ingredients (inventory). If the user selects a recipe, it should let the user know if they have the necessary ingredients at home to cook this meal. If they do, and the user clicks on "**Cook**", then it should **automatically deduct** the required ingredients from the inventory (ingredient table).

There is one important rule that must always be adhered to: The user cannot cook a meal for which he/she does not have the required ingredients.

The necessary user stories that need to be fulfilled are:

- User performs CRUD on the recipes. (**NOTE: This will affect two tables!**)
- User performs CRUD on ingredients.
- User sees all the current recipes, and can select one
- User clicks "COOK" on a selected recipe, which updates the inventory accordingly

Therefore, you will need **CRUD APIs** for all of your tables. Depending on your implementation, this includes single and/or multiple records at once.

Note: Whenever the user needs to select something, use appropriate controls, e.g. dropdowns, check boxes, multi-select lists, etc. Never should this kind of user input rely on the user correctly spelling an ingredient for instance.

Requirements

This project will be developed in **2 Sprints**. The first sprint focuses on the Python backend development, all the logic/functionality, and the deployment of the REST API services with Flask. The second sprint focuses on the UI development in JavaScript using the EJS. Both **code submissions** will happen into the **same repository** on **GitHub Classroom**. There must be a folder **'frontend'** and a folder **'backend'** in that repo, with the corresponding code bases. **Do not forget to comment and document your references.**

Each Sprint for the project is due at **midnight** of the listed due date on the schedule.

No late submission will be accepted.

Part of the submission for Sprint 2 will be a 3-5-minute long **presentation video**, giving an overview over the functionality of your application features, and explaining the technology, frameworks, etc. that you have used. This video must be made **available through a link** (for instance, recorded with MS Teams and provided is the link to MS Stream). **Do NOT submit video files on CANVAS or in your repo! Commit the link in a text file in your repo!**

Details for Sprint 1 - Deadline October 25th, 2025

For Sprint 1 you will have to design and implement all the REST API services you will need so that your app can interact with your database. (**ALL CRUD operations for all the tables.)**
These REST Services need to be implemented using the **Python Flask Framework**. You will be testing your backend code with **Postman**, without the need to have the UI pages implemented.
On CANVAS, **submit the link** to your GitHub repo. If you work as a **team of two,** one person needs to submit the **GitHub classroom link on CANVAS**, and the other person **MUST** submit a note stating the **name, student ID, and section of their partner.**
You are allowed to make modifications and changes to your Sprint 1 code even after the submission deadline, but only the last commit before the deadline will be considered for grading.

Details for Sprint 2 - Deadline December 6rd 2025

You can choose to let the user manage all necessary CRUD operations on one page (SPA) or on individual pages (multi page application). You can use any of the templates introduced in class if you like.

NOTE: Under no circumstances should the user ever see the ids used as primary keys on any of the pages! (You can, however, introduce secondary Ids of your choice of you want, for instance SKUs for ingredients).

Other Requirements

- Make sure the username and password for the database you setup are credentials you're willing to share. Do not use personal passwords for this project, which you might be using anywhere else.
- Make sure your project compiles and runs. If the project doesn't run, you will forfeit points.
- Submit early and often! **Only code that has been submitted to your repo can be graded.**
  Unexpected circumstances can occur (power outages, etc) – make sure you have commits to fall back to if need be.

- Use GitHub Classroom to setup the remote repo for this assignment

Use the provided link to create your private GitHub repo:

https://classroom.github.com/a/cCQND177

Please push your commits on a regular basis.

What to Turn in:

Commit your source code (**properly commented**) to your folders in your private repository in the GitHub classroom. **Do not zip the files together.** Your GitHub repository should show multiple meaningful commits illustrating how you worked through the project.
There will be **2 separate assignments** listed in CANVAS - one for each sprint, so that 2 separate grades can be assigned. Each assignment/sprint will be worth **50 points max**. Together, they are worth **100 points max** for the final project (not counting the extra credit).
For **both sprints,** on CANVAS simply submit the **same link** to your GitHub repository containing your project. When working in a team, submit a **project document 24h before due date for BOTH sprints**, outlining the work distribution among the team members. It should be **part of your repository** (no separate document submission on CANVAS). For **Sprint 2** just **update the project document**. Also remember, **both team members must make a CANVAS submission** as stated above, to be assigned a grade. **This applies to both Sprint 1 and Sprint 2!**
Additionally, you will need to submit the link to your **video presentation**. The video **MUST** be **accessible** for everyone **to stream/view** (verify the link before submitting). **Do NOT upload or commit a video file (.mp4, .avi, etc.)** The presentation video does not have to be narrated/showcased by both team members. However, both team members must be aware of each other's code. The video should **first showcase the functionality** of the project (without any technical details, 1-2 minutes), followed by a **short technical review** of the project, detailing the concepts, frameworks, techniques, etc used for this project (2-3 minutes). The **total length** of the video should be around **3-5 minutes**.
Do NOT submit anything via email!

Both team members will receive the **same grade for the project**.

Extra Credit

You can earn **extra credit** if you are able to deploy the Python Flask API and/or the web front-end to the cloud. **Do not attempt this until you have finished the other parts of the project.** There are many options for cloud deployment (AWS, MS Azure) and you are free to choose one that supports the technologies of this project (Python Flask backend + NodeJS/EJS based front-end). **Submit your public URL of the deployed project in a text file as part of your repository.**

Basic Grading rubric for the project

Item	Points in Sprint 1	Points in Sprint 2
Remote DB Tables setup	10	-
Code for API	35	-
Correct submission (GitHub commits, CANVAS link)	5	-
Code for UI	-	20
Code for API communication (both ways)	-	25
Correct submission (GitHub, CANVAS, including video)	-	5
Total	**50**	**50**
Extra credit for cloud deployment	**15 points**	**15 points**